Почему функции — полезны, и как их эффективно использовать?

Теперь, когда мы уже знаем, что такое **функции** и зачем они нужны, давайте более подробно рассмотрим, почему они так полезны.

Зачем использовать функции?

Начинающие программисты часто спрашивают: «А можно ли обходиться без функций и весь код помещать непосредственно в функцию main()?». Если вашего кода всего 10-20 строк, то можно. Если же серьезно, то функции предназначены для упрощения кода, а не для его усложнения. Они имеют ряд преимуществ, которые делают их чрезвычайно полезными в нетривиальных программах.

- **Структура**. Как только программы увеличиваются в размере/сложности, сохранять весь код внутри main() становится трудно. Функция — это как мини-программа, которую мы можем записать отдельно от головной программы, не заморачиваясь при этом об остальных частях кода. Это позволяет разбивать сложные задачи на более мелкие и простые, что кардинально снижает общую сложность программы.

- **Повторное использование**. После объявления функции, её можно вызывать много раз. Это позволяет избежать дублирования кода и сводит к минимуму вероятность возникновения ошибок при копировании/вставке кода. Функции также могут использоваться и в других программах, уменьшая объем кода, который нужно писать с нуля каждый раз.

- **Тестирование**. Поскольку функции убирают лишний код, то и тестировать его становится проще. А так как функция — это самостоятельная единица, то нам достаточно протестировать её один раз, чтобы убедиться в её работоспособности, а затем мы можем её повторно использовать много раз без необходимости проводить тестирование (до тех пор, пока не внесем изменения в эту функцию).

- **Модернизация**. Когда нужно внести изменения в программу или расширить её функционал, то функции являются отличным вариантом. С их помощью можно внести изменения в одном месте, чтобы они работали везде.

- **Абстракция**. Для того, чтобы использовать функцию, нам нужно знать её имя, данные ввода, данные вывода и где эта функция находится. Нам не нужно знать, как она работает. Это очень полезно для написания кода, понятного другим (например, Стандартная библиотека C++ и всё, что в ней находится, созданы по этому принципу).

Каждый раз, при вызове **std::cin или std::cout** для ввода или вывода данных, мы используем функцию из Стандартной библиотеки C++, которая соответствует всем вышеперечисленным концепциям.

Эффективное использование функций

Одной из наиболее распространенных проблем, с которой сталкиваются новички, является понимание того, где, когда и как эффективно использовать функции. Вот **несколько основных рекомендаций при написании функций**:

- *Рекомендация №1:* Код, который появляется более одного раза в программе, лучше переписать в виде функции. Например, если мы получаем данные от пользователя несколько раз одним и тем же способом, то это отличный вариант для написания отдельной функции.

- *Рекомендация №2:* Код, который используется для сортировки чего-либо, лучше записать в виде отдельной функции. Например, если у нас есть список вещей, которые нужно отсортировать — пишем функцию сортировки, куда передаем несортированный список и откуда получаем отсортированный.

- *Рекомендация №3:* Функция должна выполнять одно (и только одно) задание.

- *Рекомендация №4:* Когда функция становится слишком большой, сложной или непонятной — её следует разбить на несколько подфункций. Это называется **рефакторингом кода**.

При изучении языка C++ вам предстоит написать много программ, которые будут включать следующие три подзадания:

- Получение данных от пользователя.

- Обработка данных.

- Вывод результата.

Для простых программ (менее, чем 30 строк кода) частично или все эти три подзадания можно записать в функции main(). Для более сложных программ (или просто для практики) каждое из этих трех подзаданий является хорошим вариантом, чтобы написать отдельные функции.

Новички часто комбинируют обработку ввода и вывод результата в одной функции. Тем не менее, это нарушает **правило «одного задания»**. Функция, которая обрабатывает значение, должна возвращать его в caller, а дальше уже пускай caller сам решает, что ему с ним делать.